Solar Panel Image Capture and Condition Mapping

Mohammed Rayan

Technical report submitted for the Sunnybotics internship challenge

July 2026

Repository: https://github.com/mohammedryn/Sunnybotics---solar-panel-condition-mapping

***Abstract*—This report describes a prototype that ingests solar panel images, links each one to a location and panel identity despite GPS error, classifies visible condition, and produces a prioritized, exportable result. The pipeline runs in two modes: a fully simulated mission with GPS, odometry, and route telemetry, and a run against Sunnybotics' own sample photographs, most of which carry their own per-photo GPS EXIF but none of which carry route or panel-level ground truth. On the simulated data, the panel-association logic never misidentifies a panel, even in the half of cases where it flags its own answer as ambiguous, and a zero-shot condition classifier reaches 86.3% accuracy across five categories. Running that same classifier against real photographs is a deliberate transfer test, and it surfaces exactly the gap you would expect from thresholds calibrated only on clean synthetic renders: all 129 real images come back as uncertain rather than a wrong answer. That gap motivated a second, supervised classifier on the same features, reaching 87.6% cross-validated accuracy (Wilson 95% CI 80.8–92.2%) telling clean from damaged on real data. Both stages, the diagnostic and the fix, are reported here, because a system that only shows its successes is not one anyone should trust with thousands of panels.**

I. Introduction

Sunnybotics' brief asks for a system that can answer three questions about every photo a robot takes: where was it taken, what condition is the panel in, and what evidence backs that up. I built a five-stage pipeline for this: ingest, associate, analyze, score, and export, run with a single command (`run_pipeline.py`) taking either a `--synthetic` or `--external` flag.

Synthetic mode is a full simulated mission: a small farm layout, five rows of ten panels, two robot passes, and generated GPS, odometry, and route telemetry for 100 captures, with a handful deliberately broken so the pipeline has something real to fail on. External mode runs the identical five stages against the 129 real sample photographs Sunnybotics sent partway through the assignment window. Most carry their own GPS EXIF, but there is no route or panel-level ground truth, only a clean/damaged folder split, so I treat this mode as a transfer test for the visual-analysis stage, not a metadata benchmark. Neither mode touches the other's data or output directories.

II. Spatial Association Under GPS Error

Sunnybotics' GPS budget is 1–3 m of error, and a panel is only about 2.2 m wide, so a single fix cannot say on its own which panel a photo belongs to. I fuse GPS with odometry using a small Kalman filter, one per route pass, tracking a single number: distance travelled along the row. Odometry drives the prediction step (σ=0.05 m per step); GPS corrects it when available, with jitter around 0.4 m per fix, plus an extra term for the roughly 1 m systematic bias a receiver can carry across a whole mission that ordinary filter variance never sees on its own.

The fused distance becomes a posterior over the row's known panel positions via softmax, so panel identity comes out as a probability rather than a lookup. Four sensor situations are handled explicitly: GPS and odometry both healthy, GPS missing (predict-only), odometry looking wrong while GPS is fine (the reported step is distrusted and the estimate leans on GPS with inflated uncertainty instead), and both failing together, which is marked unresolvable rather than guessed at.

Four checks can still downgrade a result to ambiguous: GPS and odometry disagreeing across a pass, the estimate drifting past the midpoint into the next row, low overall confidence, or the top two candidate panels sitting too close to call. On the synthetic set, where the true answer is known, 49 of 100 captures were matched with confidence and 51 were flagged ambiguous, and in both groups the top guess turned out correct. That is the result I would defend hardest: the system recognizes the cases where it genuinely does not have enough signal to be sure.

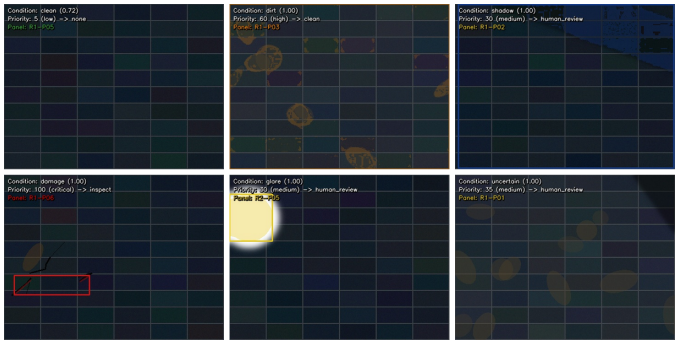

Fig. 1. One annotated synthetic output per required category. Condition, confidence, priority band, and predicted panel are burned into every processed image.

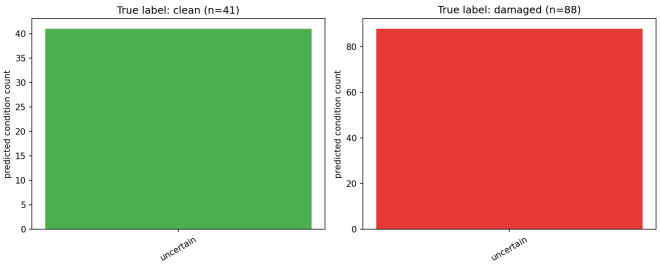

Fig. 2. Zero-shot rule classifier against all 129 real photographs. Every prediction lands on uncertain, for both true-clean and true-damaged images alike.

One limitation worth stating plainly: this association logic is validated only against my own simulator's noise model. Most real photos carry their own GPS EXIF, but there is no route structure or panel-level ground truth to match a coordinate against, so it still cannot exercise this part of the system by itself. No public dataset provides panel-level GPS ground truth at solar-farm scale either, so real-world validation of this specific piece would need either Sunnybotics' own field logs or a dedicated data-collection effort, the natural next step if this continues past the assignment.

III. Condition Analysis

Every image is reduced to ten numbers before any decision is made: area ratios for dirt, shadow, and glare, a line-density score for cracks, a blur measure, and a few others, all computed by a separate feature-extraction step so the classifier never touches raw pixels directly. Confidence per issue is a ratio against its calibrated threshold, half credit at the boundary and full credit at twice it. When two or more issues clear threshold and the gap between the top two is small, the result is called uncertain rather than picking a winner arbitrarily.

On synthetic data, where ground truth exists, this reaches 86.3% overall: clean and glare both land at 100%, damage at 88.9%, and dirt and shadow trail at 75.7% and 75.0%, the two conditions closest to each other visually.

The natural next question is whether this same zero-shot classifier transfers to real photographs, so I ran it, unmodified, against all 129 real images Sunnybotics sent. It does not, in a specific way: every image comes back uncertain rather than a wrong label, since real JPEGs carry texture and lighting noise on every frame, clearing every threshold at once and leaving nothing for the tie-break to decide between. That result is the reason the supervised classifier below exists.

I tried recalibrating thresholds per batch, using each image's own feature percentiles instead of the fixed synthetic values. That made predictions non-

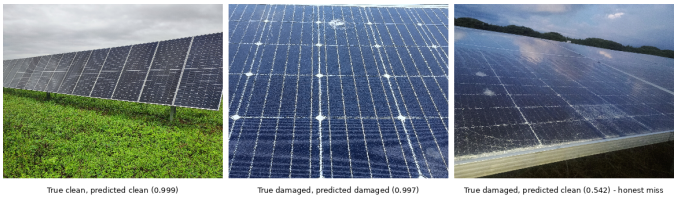

Fig. 3. Supervised classifier on real photographs: a confident correct clean call, a confident correct damaged call, and an honest miss on a soiled panel Sunnybotics labels damaged.

TABLE I
PRIORITY SCORE BY TRIGGER.

Trigger	Score rule	Action
Unusable image	flat 70	recapture
Damage (any conf.)	$70 + 30 \cdot \text{conf}_{\text{dmg}}$	inspect
Uncertain	flat 35	human_review
Dirt (dominant)	$20 + 55 \cdot \overline{\text{area}}$	clean
Shadow / glare	flat 30	human_review
Clean	flat 5	none

degenerate, but the resulting accuracy, 23.9%, was worse than simply guessing the majority class (73.9% at the time), so I reverted it rather than count it as progress.

What worked instead was a supervised classifier, logistic regression, five-fold cross-validated, standardized only within each training fold, on the same ten features. It reaches 87.6% accuracy (Wilson 95% CI 80.8–92.2%) telling clean from damaged. Fig. 3 shows three examples: clean called clean at 0.999, an impact crack called damaged at 0.997, and a genuine miss, a dust-streaked panel Sunnybotics labels *damaged*, called clean at 0.542, essentially a coin flip. I read that last case as informative, not embarrassing: it suggests the damage label covers soiling as well as breakage, a detail I would not have found without looking at the actual photo.

IV. Priority Scoring

The score branches on the full set of detected issues, not only whichever condition the classifier decided was dominant. A secondary damage signal, even a weak one, should not get buried under a more confident dirt or shadow reading, since missing a real crack is a worse mistake than an extra false alarm.

Dirt scores in proportion to the measured area of soiling rather than classifier confidence: a small confident patch and a large uncertain one call for different responses. Damage floors high regardless of what else is in the frame, since a missed structural problem costs more than an unnecessary inspection. The score is independent of how confident the panel association was; a high-priority finding on an ambiguous panel ID gets flagged in the reason text rather than folded into the score itself.

V. Assumptions

A few things are worth stating plainly. Synthetic coordinates come from a farm layout projected into local east-north-up coordinates, with GPS jitter, bias, and odometry noise added on top, documented in `farm_layout.py` and `simulate_mission.py` rather than presented as real telemetry. The external dataset carries no route or panel-level ground truth despite per-photo GPS EXIF on most images; ingestion, association, and export all check which columns are present before deciding what is required. Detection thresholds are calibrated once, on synthetic renders, and used unchanged in both modes, a deliberate transfer test, and Section VI is what happens when it is asked to do more than it can.

VI. Failure Cases and Limitations

Two separate mechanisms inject broken images into the synthetic set, and the counts differ. Five cases are injected deterministically every run: a missing file, a truncated JPEG, a GPS dropout, an out-of-range coordinate, and a capture missing its mission ID. Separately, about 3% of captures (3 images currently) get a random visual corruption: heavy blur, blackout, or overexposure. That is eight distinct broken images, not one set of five. Only five are actually rejected before condition analysis runs; the other three stay readable and instead exercise the association layer's odometry fallback. None of the eight crash the pipeline.

Beyond the injected cases, 51 of 100 synthetic associations come back ambiguous by design; Section III's real-photo zero-shot failure is the more important of the two, since a system tested only on its own simulated data would never surface it.

Fig. 4. Farm-grid dashboard, mission 1: panel ID, condition, and recommended action per cell, giving full traceability from image to location to diagnosis. It also ships as a filterable interactive HTML page.

VII. Planned Improvements

The Kalman filter currently detects GPS bias but does not correct it; adding a bias term to the state would let it estimate and remove that error instead of only widening its uncertainty around it. "Panel not visible" is approximated today by the same blackout and blur checks used elsewhere, and a dedicated occlusion detector would fit better. The supervised classifier distinguishes only clean from damaged, since that is the only label Sunnybotics' folders provide; extending it to the full five-category schema would need labeled real examples of dirt, shadow, and glare that do not yet exist.

VIII. Answers to the Required Questions

How do you guarantee that each photo is linked to the correct panel, given 1–3 m of GPS error? I do not guarantee it in the strict sense; I try to make the system honest about when it cannot be sure. Section II covers the mechanism: Kalman-fused GPS and odometry, a softmax posterior over known panel positions, and four checks that downgrade to ambiguous rather than commit. On the synthetic set, the top-1 guess was correct 100% of the time, in both the matched and ambiguous groups.

How do you detect that an image is unusable? Three cheap checks run before condition analysis: a forced full decode with PIL's `.load()` rather than `.verify()`, which I found empirically does not catch a truncated file; a pixel standard-deviation check for blackout or overexposure; and a Laplacian-variance check for blur. There is no dedicated "panel not visible" detector yet, a real gap noted in Section VII.

How would you tell real dirt apart from a temporary shadow? Dirt and shadow are measured as two separate features from the start. Where the same panel is captured across two missions with a confident association in both, that repetition helps: a signal present in only one mission is more likely a shadow, one present in both is more likely persistent dirt, though this only adjusts confidence and never overrides the single-image classification. Real photographs lack that multi-mission structure, so there the same two measurements instead feed the supervised classifier's learned weighting.

What metrics would you use without complete ground truth? Report a range rather than a single number: the Wilson interval on the real classifier, 87.6% [80.8, 92.2], matters more than the point estimate at this sample size. Track the abstention rate (ambiguous or uncertain, 51% plus 14% in synthetic mode) as a metric in its own right, since a well-calibrated system should decline exactly where it is genuinely unsure. And check whether confidence means anything by stratifying accuracy by it: on real data, the top 20% most confident predictions were 96.2% accurate against 76.9% for the bottom 20%.

Which parts would run on the robot versus in the cloud? Capture, GPS/odometry fusion, panel association, and unusable-image screening belong on the robot: low latency, no dependence on connectivity, and a bad frame gets retaken immediately rather than discovered hours later. Feature extraction, classification, the cross-mission temporal check, and the export/dashboard layer fit the cloud better, benefiting from centralized retraining as labeled data accumulates.

What is the biggest technical risk, and how would this scale to thousands of panels? The biggest risk is exactly what Section III shows: thresholds calibrated on one image distribution do not transfer to real optical conditions. At scale that becomes a large backlog of uncertain results rather than confidently wrong ones, a safer failure mode but still not a free one. The path I would take is the supervised classifier validated here: per-image inference parallelizes trivially across a farm, and the Kalman association runs in constant time per capture, so cost scales with feature-extraction compute and storage, not algorithmic complexity.

IX. Validation and Additional Work

Forty automated tests cover the pipeline, the external-dataset adapter, and the supervised classifier, and both modes reproduce byte-identical output on rerun. The cross-validation tests guarding against label leakage were mutation-tested: I reintroduced two leaks (held-out labels passed into scoring; fitting on the full dataset instead of the training fold) one at a time, confirmed each was caught, then reverted both, the difference between a test that looks right and one I have watched fail.